

# OPIPE-01: OpenPipeline as a Multi-Scope Platform

---

**Series:** OPIPE — OpenPipeline Beyond Logs | **Notebook:** 1 of 6 | **Created:** March 2026 | **Last Updated:** 09/28/2026

## Beyond Logs: Processing Spans, Metrics, and Events at Ingestion

---

OpenPipeline is often introduced as a log processing framework — and logs are indeed the most common entry point. But OpenPipeline operates on many **configuration scopes** — logs, spans, metrics, events, business events, security events and more — each with its own pipelines, routing rules, and extraction capabilities. This notebook builds intuition for how the platform works across all scopes and why pipeline design matters.

### Companion Series

- **OPLOGS** — Deep-dive into log processing: **OPLOGS-01: OpenPipeline Fundamentals**
- **OPMIG** — Migrating from classic logs? Start with **OPMIG-01: Why Migrate**
- **SPANS** — Querying and analyzing distributed traces: **SPANS-01: Fundamentals**

---

## Table of Contents

---

- [1. The OpenPipeline Scopes](#)
- [2. Shared Architecture Across Scopes](#)
- [3. The Default Pipeline Anti-Pattern](#)
- [4. Pipeline Design Principles](#)
- [5. Processing Groups: Conditional Logic Within a Pipeline](#)

- 6. [Security Context Across Scopes](#)
- 7. [Ingestion-Time vs. Query-Time Processing](#)
- 8. [Summary](#)
- 9. [Next Steps](#)
- 10. [References](#)

## Prerequisites

| Requirement           | Details                                                                                                                                                                       |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dynatrace Environment | SaaS with Grail enabled                                                                                                                                                       |
| Permissions           | <code>storage:logs:read</code> , <code>storage:spans:read</code> , <code>storage:metrics:read</code> , <code>storage:events:read</code> , <code>storage:bizevents:read</code> |
| Recommended           | Familiarity with <b>OPLOGS-01</b> (log pipeline basics)                                                                                                                       |

## Sprint 1.337 (April 2026): OneAgent Primary Fields/Tags as Routing Keys

Sprint 1.337 SaaS landed a major OpenPipeline-relevant change: OneAgent now enriches **all** telemetry (spans, metrics, logs, business events, Smartscape entities) at the source with standardized **primary fields** and customer-defined **primary tags** as top-level attributes on Latest Dynatrace tenants.

**Primary fields** (Semantic Dictionary-defined):

| Field                            | Purpose                                                           |
|----------------------------------|-------------------------------------------------------------------|
| <code>dt.security_context</code> | ABAC scope for IAM policies; also drives bucket-routing decisions |
| <code>dt.cost.costcenter</code>  | Cost allocation tag; routes spend to org units                    |
| <code>dt.cost.product</code>     | Product-line attribution for cost rollups                         |

**Primary tags** (customer-defined): set during OneAgent install via `oneagentctl --set-host-tag="primary_tags.<key>=<value>"` — the `primary_tags.` prefix must be written

explicitly (it is never added automatically); e.g., `primary_tags.team` , `primary_tags.env` , `primary_tags.app` , `primary_tags.data_classification` . Land as `primary_tags.<key>` on every signal, up to 20 primary tags per host or process — excess tags are silently dropped without a warning.

**Pipeline implication for cross-scope design:**

- 1. **Routing rules** can dispatch on primary fields directly without parse processors. A route's matching condition is DQL, and a route sends records to a **pipeline**, not to a bucket:

| Route matcher (DQL)                                     | Target pipeline                                                                 |
|---------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>dt.cost.costcenter == "cc-1234"</code>            | <code>finance-logs</code>                                                       |
| <code>matchesValue(dt.security_context, "*pci*")</code> | <code>pci-audit</code> (whose Bucket assignment stage picks the 365-day bucket) |

The bucket is chosen later, by the target pipeline's **Bucket assignment** stage. Write substring tests as `matchesValue(field, "*text*")` , not `contains()` : OpenPipeline matchers accept only a subset of DQL functions, and a condition written with `contains()` or `in()` is rejected with the verifier error `The function contains() isn't enabled.` `matchesValue` is case-insensitive and also matches any element when `dt.security_context` holds an array value (see the array rule in §6).

- 2. **Cross-scope consistency:** because primary fields/tags appear on logs, spans, metrics, AND business events, queries that join across scopes (OPIPE-06 cross-scope design patterns) can correlate without scope-specific lookup tables.
- 3. **Non-OneAgent sources** (raw syslog, third-party log shippers, OTLP-via-collector) still need OpenPipeline `enrichment` processors to surface the same fields. Document this split in your standard.

**Update (June 2026) — primary tags are now formally documented.** Dynatrace published a dedicated [Primary tags \(DT docs\)](#) page that canonicalizes what sprint 1.337 introduced. Two points matter for pipeline design: (1) **OpenPipeline itself is a documented primary-tag source** — `primary_tags.*` values can be *"derived or transformed from any incoming field at ingest-processing time"*, which closes the non-OneAgent-sources gap in point 3 above: instead of generic enrichment processors producing ad-hoc fields, derive proper `primary_tags.<key>` values so downstream routing and bucket assignment treat agent and agentless data identically. (2) The other documented sources are OneAgent, Kubernetes

annotations ( `metadata.dynatrace.com/primary_tags.<key>` ), cloud-provider tags, OpenTelemetry resource attributes, and host or process metadata. Primary tags are "available before data enters the processing pipeline," which is exactly what makes them usable in routing conditions like the examples above.

**Sprint-337 also added recommended-field suggestions to extraction processors** — the UI flags permission-relevant fields and Smartscape identifiers in the field-promotion dialog, preventing accidental promotion of sensitive content. Existing extraction processors keep working unchanged.

## 1. The OpenPipeline Scopes

OpenPipeline processes data through independent **configuration scopes** — logs, spans, metrics, generic events, Davis events and problems, SDLC events, business events, security events, system events (limited support), Smartscape events (limited support), user events, and user sessions. Each scope has its own set of pipelines, routing rules, processors, and storage targets. They share the same architecture but operate on different data types. This series covers the six you will configure most:

| Scope                  | Data Object                  | Primary Use Case                                    | Key Fields                                                             |
|------------------------|------------------------------|-----------------------------------------------------|------------------------------------------------------------------------|
| <b>Logs</b>            | <code>logs</code>            | Application and infrastructure log records          | <code>content</code> , <code>loglevel</code> , <code>log.source</code> |
| <b>Spans</b>           | <code>spans</code>           | Distributed trace spans from services               | <code>span.kind</code> , <code>trace.id</code> , <code>duration</code> |
| <b>Metrics</b>         | <code>timeseries</code>      | Time-series measurements from agents and extensions | <code>metric.key</code> , <code>dt.entity.*</code>                     |
| <b>Events</b>          | <code>events</code>          | Lifecycle and configuration change events           | <code>event.kind</code> , <code>event.type</code>                      |
| <b>Business Events</b> | <code>bizevents</code>       | Business transactions and user actions              | <code>event.type</code> , <code>event.provider</code>                  |
| <b>Security Events</b> | <code>security.events</code> | Vulnerability, detection, and compliance findings   | <code>event.kind</code> , <code>event.type</code>                      |

Sources: [OpenPipeline data flow \(DT docs\)](#) — configuration scope list.

## What This Means in Practice

Each scope is configured **independently** in the Dynatrace UI under **OpenPipeline** settings. A processing rule you create for logs has no effect on spans. A routing rule for business events does not apply to metrics. This independence is a feature — it means you can evolve each scope's pipeline at its own pace without risk to other data types.

## Discovering Your Active Scopes

The following queries show what data is flowing through each scope in your environment.

```
// Log pipelines: volume by pipeline and bucket
fetch logs, from:-1h
| summarize log_count = count(), by:{dt.openpipeline.pipelines,
dt.system.bucket}
| sort log_count desc
```

```
// Span pipelines: volume by pipeline and bucket
fetch spans, from:-1h
| summarize span_count = count(), by:{dt.openpipeline.pipelines,
dt.system.bucket}
| sort span_count desc
```

```
// Business event pipelines: volume by provider and type
fetch bizevents, from:-1h
| summarize event_count = count(), by:{event.provider, event.type}
| sort event_count desc
```

## 2. Shared Architecture Across Scopes

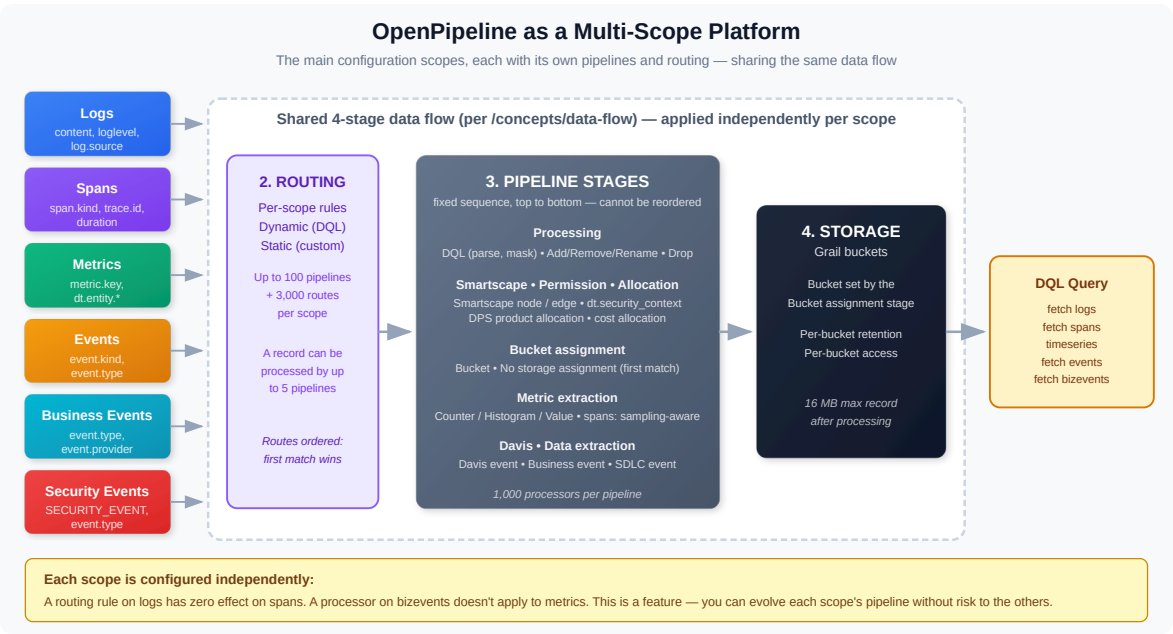
### Sprint 1.345 (August 2026): Temporary Fields ( `dt.temp*` )

A field whose name is prefixed `dt.temp` lives for the duration of pipeline processing and is **discarded before the record is written to Grail**. That closes a long-standing gap in the extraction pattern this series teaches: previously, an intermediate value needed for

metric, event, or bizevent extraction had to be written onto the record — where it persisted, consumed storage, and changed the shape of the stored data for every downstream consumer.

| Property      | Behaviour                                                              |
|---------------|------------------------------------------------------------------------|
| Lifetime      | Available across all processing stages; dropped at storage             |
| Persistence   | Never written to Grail — invisible to <code>fetch</code>               |
| Cost          | Not persisted, so it adds no ingest or retention charge                |
| Enablement    | None — the <code>dt.temp</code> prefix alone marks the field temporary |
| Scope support | Every scope with a Processing stage                                    |

Use it for intermediate calculations feeding extraction. Do **not** use it for enrichment you intend to query later — the field will not be there. SaaS 1.345 released 08/11/2026 with a **staged tenant rollout**; verify availability in your tenant before relying on the prefix, and note that a `dt.temp` field written on a tenant that does not yet support the semantics would simply persist as an ordinary attribute.



Every scope follows the same **data flow** (Ingest → Routing → Processing → Storage), with optional pre-processing on custom sources. Inside a pipeline, processing is not one step: it runs as a **fixed sequence of stages**, of which the stage named *Processing* is only the first. The flow is identical in concept across scopes — only the data types and available processors differ.

| Stage              | Purpose                                                                                                                                                                                                                                                                                                                                                             | Logs Example                                                                                                  | Spans Example                                                                                  |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>1. Ingest</b>   | Receive records from a built-in / ready-made / custom source                                                                                                                                                                                                                                                                                                        | OneAgent log ingest                                                                                           | OTLP span ingest                                                                               |
| <b>2. Routing</b>  | Direct data to the right pipeline (dynamic DQL match or static for custom sources)                                                                                                                                                                                                                                                                                  | Match by <code>log.source</code> or <code>k8s.namespace.name</code>                                           | Match by <code>span.kind</code> or <code>service.name</code>                                   |
| <b>3. Pipeline</b> | A fixed sequence of stages: <b>Processing</b> (DQL, add / remove / rename fields, drop record, GeoIP and inline lookup) → <b>Smartscape node</b> → <b>Smartscape edge</b> → <b>Permission</b> (security context) → <b>Product allocation</b> → <b>Cost allocation</b> → <b>Bucket assignment</b> → <b>Metric extraction</b> → <b>Davis</b> → <b>Data extraction</b> | Mask credit card numbers, drop debug, parse JSON (Processing); extract error-rate metrics (Metric extraction) | Mask PII, drop health-check spans (Processing); sampling-aware RED metrics (Metric extraction) |
| <b>4. Storage</b>  | Persist to the Grail bucket chosen in the Bucket assignment stage (or skip with No storage assignment)                                                                                                                                                                                                                                                              | Send to <code>app_logs</code> bucket                                                                          | Send to <code>trace_data</code> bucket                                                         |

**Stage order is fixed.** Per *Processing in OpenPipeline*, "The sequence of stages is fixed for all pipelines and cannot be modified." Masking, parsing, and dropping happen in the first stage (Processing); Metric extraction, Davis, and Data extraction run **after** Bucket assignment. Within a stage, processors run in the order you list them, and each stage executes either all matching processors or only the first match.

**Sources:** [Processing in OpenPipeline \(DT docs\)](#) — stage table, in execution order.

## Sprint 1.346 (August 2026): `dt.bindplane.*` Is a Reserved Namespace

Verbatim: "The `dt.bindplane.*` namespace is now reserved exclusively for Bindplane. Dynatrace uses it to populate fields such as `dt.bindplane.project`, `dt.bindplane.fleet`, and `dt.bindplane.configuration`" — to keep those fields populated consistently.

The practical rule for pipeline authors is the same one that governs every `dt.*` namespace: **do not write into it from a processor**. If a processing rule of yours adds or overwrites a `dt.bindplane.*` field, move it to a namespace you own before the reservation takes effect, or the platform-populated value and yours will contend. This is worth a grep of your existing pipelines rather than an assumption — the namespace was not reserved before, so nothing stopped a rule from using it.

SaaS 1.346 released 08/25/2026 with a **staged tenant rollout** from the same date; verify against your own tenant.

### The Key Insight

If you understand how to build a log pipeline (route to a pipeline → mask, drop, and transform in Processing → set permissions and cost allocation → assign a bucket → extract metrics and events), you already understand how to build a span pipeline or an event pipeline. The concepts transfer directly — only the field names and processor options change.

### Processors Available Per Scope

Not every processor is available in every scope. The following table shows key differences (refresh against [Processing in OpenPipeline \(DT docs\)](#) for the current list):

| Processor                                      | Logs | Spans | Metrics | Events | Bizevents |
|------------------------------------------------|------|-------|---------|--------|-----------|
| DPL parsing                                    | Yes  | Yes   | —       | Yes    | Yes       |
| Field enrichment (Add / Remove / Rename / DQL) | Yes  | Yes   | Yes     | Yes    | Yes       |

| Processor                                                                         | Logs | Spans                            | Metrics | Events | Bizevents |
|-----------------------------------------------------------------------------------|------|----------------------------------|---------|--------|-----------|
| Metric extraction (Counter / Value / Histogram Preview)                           | Yes  | Yes<br>(sampling-aware variants) | —       | Yes    | Yes       |
| Smartscape node / edge                                                            | Yes  | Yes                              | Yes     | Yes    | Yes       |
| Event extraction (Business / SDLC event in Data extraction; Davis event in Davis) | Yes  | Yes                              | —       | Yes    | Yes       |
| Masking (DQL processor, Processing stage)                                         | Yes  | Yes                              | Yes     | Yes    | Yes       |
| Inline lookup (Processing stage)                                                  | Yes  | Yes                              | Yes     | Yes    | Yes       |
| GeoIP lookup (Processing stage, Early Access)                                     | Yes  | Yes                              | Yes     | Yes    | Yes       |
| Drop record                                                                       | Yes  | Yes                              | Yes     | Yes    | Yes       |
| Bucket assignment / No storage assignment                                         | Yes  | Yes                              | —       | Yes    | Yes       |
| Set dt.security_context                                                           | Yes  | Yes                              | Yes     | Yes    | Yes       |
| DPS Cost Allocation (Cost Center / Product)                                       | Yes  | Yes                              | Yes     | —      | —         |

## OneAgent Attribute Enrichment (OneAgent 1.333+)

**Requires:** OneAgent version **1.333** or later

OneAgent can enrich **all telemetry signals** (metrics, spans, logs, events, entities) with custom metadata at the source — before data reaches the Dynatrace platform. This is more efficient than server-side tagging (auto-tags) because enrichment happens on the host and propagates to all Smartscape nodes.

**Primary Fields** (standardized from Semantic Dictionary):

- `dt.security_context` — data governance and access control

- `dt.cost.costcenter` — cost allocation
- `dt.cost.product` — product attribution

**Primary Tags** (custom key-value pairs):

- `primary_tags.environment` — environment identification (production, staging, etc.)
- `primary_tags.team` — team ownership
- `primary_tags.business_unit` — organizational unit

### Configuration:

```
# Set during OneAgent installation
Dynatrace-OneAgent-Linux.sh --set-host-tag="primary_tags.environment=production" --set-host-tag="dt.security_context=confidential"

# Set on existing agents via oneagentctl
oneagentctl --set-host-tag="primary_tags.environment=production"
oneagentctl --set-host-tag="dt.cost.costcenter=12345"

# Per-process via environment variable (overrides host-level)
DT_TAGS="primary_tags.team=platform primary_tags.environment=production"
```

### Benefits over auto-tagging:

| Aspect            | Auto-Tags (Server-Side)        | Attribute Enrichment (Agent-Side)                                     |
|-------------------|--------------------------------|-----------------------------------------------------------------------|
| When applied      | After data arrives at platform | At the source, before transmission                                    |
| Scope             | Entity tags only               | All signals: metrics, spans, logs, events, entities                   |
| Grail integration | Limited                        | Full — feeds OpenPipeline routing, bucket assignment, permissions     |
| Cost allocation   | Not supported                  | <code>dt.cost.costcenter</code> , <code>dt.cost.product</code> fields |
| Security context  | Not supported                  | <code>dt.security_context</code> for data governance                  |

See: [Primary Grail fields and tags enrichment through OneAgent \(DT docs\)](#)

## 3. The Default Pipeline Anti-Pattern

Every OpenPipeline scope has a built-in **Default Pipeline** — for logs and business events, only where the Classic pipeline is not available. It is a catch-all: any data that does not match a custom pipeline's routing rules flows through the default pipeline and lands in the default bucket (e.g., `default_logs`, `default_spans`).

This is convenient for getting started. It becomes a serious problem at scale — Dynatrace's own guidance is to *"limit the use of the default pipeline to monitoring unassigned incoming data"*.

**Sources:** [Processing in OpenPipeline \(DT docs\)](#) — *"It's not available for log and business event configuration scopes where the Classic pipeline is available."*

## Why "Everything in Default" Fails

| Problem                     | Impact                                                                                                                                                                                                                |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Query performance</b>    | Every DQL query against the default bucket scans ALL data — application logs mixed with infrastructure noise, health checks mixed with business transactions. The 500 GB scan limit becomes a wall.                   |
| <b>Cost</b>                 | No differentiated retention. You pay to store debug-level container stdout at the same retention tier as critical security audit logs.                                                                                |
| <b>Security</b>             | No <code>dt.security_context</code> separation. Without dedicated pipelines, you cannot apply IAM policies to restrict who sees which data. Finance team logs are visible to infrastructure engineers and vice versa. |
| <b>Blast radius</b>         | A misconfigured processing rule (bad parsing regex, aggressive drop filter) in the default pipeline affects ALL data flowing through it. One mistake breaks everything.                                               |
| <b>No metric extraction</b> | You cannot extract targeted metrics from specific log patterns when everything is in one undifferentiated stream. Metric extraction rules become overly complex or extract noise.                                     |
| <b>Debugging</b>            | When something goes wrong, you cannot tell which data source caused the issue. Pipeline-level monitoring ( <code>dt.openpipeline.pipelines</code> ) shows a single pipeline name for everything.                      |

## The Telltale Signs

Run the following queries to check if your environment has this problem.

```
// Check: What percentage of logs land in the default bucket?
fetch logs, from:-1h
| summarize total = count(),
    default_count = countIf(dt.system.bucket == "default_logs")
| fieldsAdd default_pct = round(toDouble(default_count) / toDouble(total) *
100, decimals: 1)
| fieldsAdd assessment = if(default_pct > 80, then: "CRITICAL: Most data in
default bucket",
    else: if(default_pct > 50, then: "WARNING: Majority in default bucket",
    else: "OK: Data is distributed across buckets"))
```

```
// Check: How many distinct pipelines are processing your logs?
fetch logs, from:-1h
| summarize pipeline_count = countDistinct(dt.openpipeline.pipelines),
    bucket_count = countDistinct(dt.system.bucket),
    source_count = countDistinct(dt.openpipeline.source)
| fieldsAdd assessment = if(pipeline_count <= 1, then: "CRITICAL: Only
default pipeline in use",
    else: if(pipeline_count < 3, then: "WARNING: Very few pipelines
configured",
    else: "OK: Multiple pipelines configured"))
```

```
// Visualize: Volume distribution across buckets (are they balanced?)
//
// The grand total must be computed BEFORE the rows are split apart. After a
grouped
// summarize, log_count is a scalar per row – not an array – so
arraySum(log_count) is
// a type mismatch. The query still returns ok:true and rows, and every
pct_of_total
// reads null, with only a DATATYPE_MISMATCH warning to signal it. Collect the
rows
// into one array, total them there, then expand back to one row per bucket.
fetch logs, from:-1h
| summarize log_count = count(), by:{dt.system.bucket}
| summarize rows = collectArray(record(bucket = dt.system.bucket, log_count =
log_count)),
    grand_total = sum(log_count)
| expand rows
| fieldsAdd bucket = rows[bucket], log_count = rows[log_count]
| fieldsAdd pct_of_total = round(100.0 * log_count / grand_total, decimals: 1)
| fields bucket, log_count, pct_of_total
| sort log_count desc
```

```
// Same check for spans: are they all in default_spans?
fetch spans, from:-1h
| summarize total = count(),
    default_count = countIf(dt.system.bucket == "default_spans")
| fieldsAdd default_pct = round(toDouble(default_count) / toDouble(total) *
100, decimals: 1)
| fieldsAdd assessment = if(default_pct > 80, then: "CRITICAL: Most spans
in default bucket",
    else: if(default_pct > 50, then: "WARNING: Majority in default bucket",
    else: "OK: Spans are distributed across buckets"))
```

## 4. Pipeline Design Principles

The goal is to move from "everything in default" to a **purpose-driven pipeline architecture**. Here are the principles:

### Principle 1: One Pipeline Per Source Type

Create separate pipelines based on the nature of the data, not the team that owns it.

| Pipeline         | Routing Condition                                                            | Target Bucket  | Retention |
|------------------|------------------------------------------------------------------------------|----------------|-----------|
| kubernetes-infra | k8s.namespace.name exists AND<br>log.source == "Container<br>Output"         | k8s_infra_logs | 14 days   |
| application-logs | log.source == "Log file" AND<br>k8s.namespace.name matches app<br>namespaces | app_logs       | 35 days   |
| security-audit   | loglevel == "AUDIT" OR<br>log.source matches security<br>sources             | security_audit | 365 days  |
| api-ingestion    | dt.openpipeline.source ==<br>"/api/v2/logs/ingest"                           | external_logs  | 35 days   |
| Default Pipeline | Everything else (catch-all)                                                  | default_logs   | 14 days   |

## Principle 2: Filter Early, Extract Targeted

Each pipeline should drop noise **before** storage and extract **only** the metrics relevant to that data type:

- **kubernetes-infra pipeline:** Drop health check logs, extract pod restart counts
- **application-logs pipeline:** Drop DEBUG in production, extract error rates by service
- **security-audit pipeline:** Never drop, never sample — extract compliance event counts

## Principle 3: Differentiate Retention

Different data has different value over time:

| Data Type              | Suggested Retention | Rationale                               |
|------------------------|---------------------|-----------------------------------------|
| Debug/container stdout | 7-14 days           | High volume, short-term troubleshooting |
| Application logs       | 35 days             | Standard operational window             |
| Security/audit logs    | 365+ days           | Compliance requirements (SOC 2, HIPAA)  |
| Business events        | 90-365 days         | Trend analysis, quarterly reporting     |

## Principle 4: Route Ordering Matters

Pipeline routing rules are evaluated **in order** — **first match wins**. Place specific rules before general ones:

1. Security audit logs (most specific, highest value)
2. Application logs by namespace
3. Kubernetes infrastructure
4. API-ingested external logs
5. Default pipeline (catch-all for anything unmatched)

## Applying This to Spans

The same principles apply to the spans scope:

| Pipeline         | Routing Condition                                                | Target Bucket  |
|------------------|------------------------------------------------------------------|----------------|
| frontend-traces  | span.kind == "server" AND service.name matches frontend services | frontend_spans |
| backend-traces   | span.kind == "server" AND service.name matches backend services  | backend_spans  |
| database-spans   | span.kind == "client" AND db.system exists                       | db_spans       |
| Default Pipeline | Everything else                                                  | default_spans  |

```
// Inventory: What log sources exist and how much volume do they produce?
// Use this to plan your pipeline routing rules
fetch logs, from:-24h
| summarize log_count = count(),
  unique_hosts = countDistinct(dt.entity.host),
  by:{dt.openpipeline.source, log.source}
| sort log_count desc
| limit 20
```

```
// Inventory: Kubernetes namespace volume (for pipeline routing decisions)
fetch logs, from:-24h
| filter isNotNull(k8s.namespace.name)
| summarize log_count = count(), by:{k8s.namespace.name}
| sort log_count desc
| limit 20
```

```
// Inventory: Span volume by service (for span pipeline routing decisions)
fetch spans, from:-24h
| summarize span_count = count(), by:{service.name, span.kind}
| sort span_count desc
| limit 20
```

## 5. Processing Groups: Conditional Logic Within a Pipeline

Sections 3 and 4 explained why you need separate pipelines and how to design them. But what happens when a single pipeline receives data from multiple sources that need

**different processing logic** — without justifying an entirely new pipeline?

This is what **processing groups** solve.

## What Is a Processing Group?

A processing group is a named collection of processors within a pipeline stage that share a **common matching condition**. Only records that satisfy the group's matcher are processed by the group's processors. Records that don't match skip the group entirely.

```
Pipeline: application-logs
├─ Processing Stage
│   ├── Group: "java-apps"      (matcher: k8s.namespace.name == "java-prod")
│   │   ├── Processor: Parse Java stack traces
│   │   ├── Processor: Extract exception class
│   │   └─ Processor: Add team = "backend"
│   │
│   ├── Group: "nginx-access"  (matcher: log.source == "nginx")
│   │   ├── Processor: Parse access log format
│   │   ├── Processor: Extract response code + latency
│   │   └─ Processor: Add team = "platform"
│   │
│   └─ Global Processor: Add environment = "production" (no matcher – runs
on ALL records)
```

## How Processing Groups Work

| Behavior          | Detail                                                                                                                               |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Matching          | Each group has a DQL matching condition. Only records satisfying the condition enter the group.                                      |
| Inheritance       | All processors within a group inherit the group's matcher. You do not repeat the condition on each processor.                        |
| Multi-match       | If a record matches multiple groups, <b>all matching groups execute</b> . Groups are independent — they do not short-circuit.        |
| Execution order   | Processors <b>within</b> a group execute in the order they are listed.                                                               |
| Global processors | Processors placed outside any group run on <b>all records</b> unconditionally — before or after groups, depending on their position. |

| Behavior                       | Detail                                                                              |
|--------------------------------|-------------------------------------------------------------------------------------|
| <b>Available in all scopes</b> | Processing groups work in Logs, Spans, Metrics, Events, and Business Events scopes. |

**SaaS 1.344+:** OpenPipeline matchers support **Duration**-type record fields, with the operators `=`, `!=`, `>`, `>=`, `<`, and `<=` (rollout from 07/29/2026). On a tenant that has not yet reached 1.344, match on an equivalent **numeric** field instead (for example a millisecond-valued field carried alongside the record rather than the duration field itself).

## When to Use Processing Groups vs. Separate Pipelines

| Scenario                                   | Use Processing Groups                  | Use Separate Pipelines                                             |
|--------------------------------------------|----------------------------------------|--------------------------------------------------------------------|
| Same bucket destination, different parsing | <b>Yes</b> — group per log format      | No                                                                 |
| Different retention requirements           | No                                     | <b>Yes</b> — different buckets                                     |
| Different security context                 | No                                     | <b>Yes</b> — different <code>dt.security_context</code> assignment |
| Same source, multiple enrichment paths     | <b>Yes</b> — group per enrichment path | No                                                                 |
| Completely different data lifecycle        | No                                     | <b>Yes</b> — separate routing, storage, retention                  |
| Reducing pipeline proliferation            | <b>Yes</b> — consolidate related logic | N/A                                                                |
| Team-level blast radius isolation          | No                                     | <b>Yes</b> — a bad processor in one pipeline can't affect another  |

## The Decision Rule

**Use processing groups** when the data shares the same pipeline-level concerns (routing, bucket, retention, security context) but needs **different processing logic**

based on content.

**Use separate pipelines** when the data needs different **lifecycle treatment** — different buckets, different retention, different security context, or different teams owning the configuration.

## Example: Multi-Format Log Pipeline

A single `application-logs` pipeline receives logs from Java services, Node.js services, and Python services. Each has a different log format, but they all go to the same `app_logs` bucket with 35-day retention.

**Without processing groups**, you would need 3 separate pipelines — tripling the configuration surface and the number of routing rules.

**With processing groups:**

| Group                    | Matching Condition                                        | Processors                                                                                              |
|--------------------------|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>java-logs</code>   | <code>matchesValue(k8s.container.name, "java-*")</code>   | Parse log4j format, extract exception class, enrich with <code>app.framework = "java"</code>            |
| <code>nodejs-logs</code> | <code>matchesValue(k8s.container.name, "node-*")</code>   | Parse JSON structured logs, extract request ID, enrich with <code>app.framework = "nodejs"</code>       |
| <code>python-logs</code> | <code>matchesValue(k8s.container.name, "python-*")</code> | Parse Python logging format, extract traceback, enrich with <code>app.framework = "python"</code>       |
| <i>(global)</i>          | All records                                               | Add <code>dt.security_context = "app-team"</code> , drop records where <code>loglevel == "DEBUG"</code> |

The global processors handle what's common to all records. The groups handle what's specific to each format. One pipeline, one bucket, three parsing paths.

## Example: Span Enrichment by Service Category

In the Spans scope, a single pipeline processes all server spans but needs different enrichment based on service type:

| Group          | Matching Condition                                                                                              | Processors                                                                                             |
|----------------|-----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| api-services   | <code>matchesValue(http.route, "/api/*")</code>                                                                 | Add <code>span.category = "api"</code> , extract API version from path                                 |
| database-calls | <code>isNotNull(db.system)</code>                                                                               | Add <code>span.category = "database"</code> , normalize <code>db.statement</code> to remove parameters |
| messaging      | <code>span.kind == "producer" or span.kind == "consumer" ( in() is not enabled in OpenPipeline matchers)</code> | Add <code>span.category = "messaging"</code> , extract queue name                                      |

## Common Mistakes

| Mistake                                                               | Problem                                                                 | Fix                                                                                        |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Duplicating matchers on every processor                               | Verbose, error-prone, hard to maintain                                  | Use a processing group — define the matcher once                                           |
| Overlapping group matchers without intent                             | A record processed by multiple groups may get conflicting field values  | Make matchers mutually exclusive, or design for intentional multi-match                    |
| One pipeline per destination bucket when only the destination differs | Pipeline proliferation for data that is otherwise processed identically | Use conditional <b>Bucket assignment</b> processors (first match wins) inside one pipeline |
| Too many groups in one pipeline                                       | Becomes as complex as having separate pipelines                         | If you have >5-6 groups, consider splitting into separate pipelines                        |

## 6. Security Context Across Scopes

The `dt.security_context` field controls **who can see which data** through IAM policies. It is the mechanism that makes pipeline separation actionable from a governance perspective.

## How Security Context Works

When data flows through an OpenPipeline, you can enrich it with a `dt.security_context` value. IAM policies then use this value to restrict query access:

```
Data Ingestion → OpenPipeline adds dt.security_context → Grail stores it → IAM policy enforces access
```

For a complete deep-dive on security context patterns, values, and IAM policy syntax, see **ORGNZ-06: Security Context**.

## Security Context by Scope

| Scope                  | How to Set <code>dt.security_context</code>                               | Notes                                                                                                   |
|------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>Logs</b>            | OpenPipeline field enrichment processor                                   | Most common — set based on <code>k8s.namespace.name</code> , <code>log.source</code> , or parsed fields |
| <b>Spans</b>           | OpenPipeline field enrichment processor                                   | Set based on <code>service.name</code> or span attributes                                               |
| <b>Metrics</b>         | OpenPipeline field enrichment processor <b>or</b> extension configuration | See "Extension Metrics" below                                                                           |
| <b>Events</b>          | OpenPipeline field enrichment processor                                   | Set based on <code>event.type</code> or entity association                                              |
| <b>Business Events</b> | OpenPipeline field enrichment processor                                   | Set based on <code>event.provider</code> or business unit                                               |
| <b>Entities</b>        | Entity properties via Settings API                                        | Set on the entity itself, inherited by associated data                                                  |

## Extension Metrics and Security Context

Metrics ingested via **Extensions 2.0** do not automatically inherit `dt.security_context`. This is a common gap — teams deploy custom extensions to collect database metrics, cloud service metrics, or infrastructure data, and assume the security context is handled automatically.

It is not. You must explicitly configure it.

There are two approaches:

**Approach 1: OpenPipeline Metric Enrichment (Recommended)**

Configure the **Metrics** scope in OpenPipeline to add `dt.security_context` based on dimensions that identify the source:

| Routing Condition                                                                     | Security Context Value | Use Case                           |
|---------------------------------------------------------------------------------------|------------------------|------------------------------------|
| <code>metric.key</code> starts with <code>ext:com.dynatrace.extension.database</code> | "database-team"        | Database monitoring extensions     |
| <code>metric.key</code> starts with <code>ext:com.dynatrace.extension.cloud</code>    | "cloud-team"           | Cloud integration extensions       |
| <code>dt.entity.host</code> matches specific host group                               | "team-a"               | Team-scoped infrastructure metrics |

**Approach 2: Entity-Level Security Context**

Set `dt.security_context` on the **entities** that the extension monitors (hosts, process groups). Metrics associated with those entities inherit the context through entity relationships. This is less granular but simpler to manage.

**Structured context design:** For organisations with transversal teams (database, networking, OS) that need cross-application access, a flat context value (e.g., "checkout" ) does not scale. Encoding `comp:<component>/bu:<business-unit>/app:<application>` into the context string, then using `MATCH('comp:db*')` in the IAM policy, enables precise transversal access without a per-application context per team. The dimension needed for transversal slicing should come first in the string for compatibility with Classic entity `startsWith` matching. See **IAM-04: Policy Authoring** for the complete design pattern and **IAM-05: Boundary Design** for boundary examples.

**Critical:** When `dt.security_context` holds an array value, IAM policies must use the `MATCH` operator — not `=`, `STARTSWITH`, or `IN`. Array comparison with equality operators always returns false. See **ORGNZ-06** for the full rules.

```
// Check: Do your logs have security context assigned?
fetch logs, from:-1h
| summarize total = count(),
    with_context = countIf(isNotNull(dt.security_context)),
    without_context = countIf(isNull(dt.security_context))
| fieldsAdd coverage_pct = round(toDouble(with_context) / toDouble(total) *
100, decimals: 1)
```

```
// Check: Do your spans have security context assigned?
fetch spans, from:-1h
| summarize total = count(),
    with_context = countIf(isNotNull(dt.security_context)),
    without_context = countIf(isNull(dt.security_context))
| fieldsAdd coverage_pct = round(toDouble(with_context) / toDouble(total) *
100, decimals: 1)
```

## 7. Ingestion-Time vs. Query-Time Processing

A critical design decision: should you process data in OpenPipeline (at ingestion) or in DQL (at query time)? The answer depends on what you are trying to achieve.

### When to Process at Ingestion (OpenPipeline)

| Use Case                    | Why Ingestion Time                                                                                                                           |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Drop noise</b>           | Data you never want to see should never be stored. Dropping debug logs, health check spans, or synthetic test events saves cost permanently. |
| <b>Mask sensitive data</b>  | PII must be redacted before storage for compliance. You cannot "un-store" data.                                                              |
| <b>Add security context</b> | <code>dt.security_context</code> must be set at ingestion — it cannot be added retroactively.                                                |
| <b>Extract metrics</b>      | Converting log patterns or span attributes into metrics happens at ingestion. You cannot create metrics from historical data.                |
| <b>Route to buckets</b>     | Bucket assignment is permanent at ingestion time. You cannot move data between buckets after storage.                                        |

## When to Process at Query Time (DQL)

| Use Case                      | Why Query Time                                                                                                 |
|-------------------------------|----------------------------------------------------------------------------------------------------------------|
| Exploratory analysis          | When you do not yet know what you are looking for, keep the raw data and query flexibly.                       |
| Changing requirements         | If the transformation logic changes frequently, DQL queries are easier to update than pipeline configurations. |
| Ad-hoc parsing                | One-time investigations benefit from DQL <code>parse</code> rather than permanent pipeline processors.         |
| Complex joins                 | Correlating across multiple data sources is a query-time operation.                                            |
| Aggregation and visualization | Dashboard queries, trend analysis, and statistical aggregations belong in DQL.                                 |

## A Third Surface: At-Source Enrichment on OneAgent (OneAgent 1.343)

The choice above is about *where inside Dynatrace* processing happens. OneAgent 1.343 adds a placement surface **upstream of both**: OneAgent ingests enrichment configuration directly and applies tags and metadata conditionally using **DQL matcher functions** (equality, phrase, and existence checks) at the point of capture — before the record reaches OpenPipeline.

| Surface                      | Runs                         | Use it for                                                                                                                   |
|------------------------------|------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| OneAgent enrichment (1.343+) | At the source, before ingest | Attributes only knowable on the host — process, container, or host-local metadata you would otherwise reconstruct downstream |
| OpenPipeline                 | At ingestion                 | Everything in the first table above: drop, mask, security context, metric extraction, bucket routing                         |
| DQL                          | At query time                | Everything in the second table above: exploration, ad-hoc parsing, joins, aggregation                                        |

Rollout is **per agent fleet, not per tenant** — **verify the OneAgent version on the hosts concerned**; the tenant version is not the agent version, and fleets commonly lag by more than one sprint. On OneAgent 1.342 and earlier the two-surface model above is the whole picture.

**Enriching at the source does not displace OpenPipeline.** Security context, bucket routing, and masking remain OpenPipeline responsibilities — the gain is that a field is already present *when* OpenPipeline evaluates its matchers, so pipeline logic that would otherwise derive it can just match on it.

## The Decision Framework

Ask these questions in order:

1. **Is the data sensitive?** → Mask at ingestion. No exceptions.
2. **Is the data noise?** → Drop at ingestion. Do not pay to store it.
3. **Does it need access control?** → Add security context at ingestion.
4. **Do you need a metric from it?** → Extract at ingestion.
5. **Everything else?** → Store raw, process at query time.

---

## Summary

In this notebook you learned:

- **Configuration scopes** — OpenPipeline processes logs, spans, metrics, events, business events, security events, and more, each independently
- **Shared architecture** — All scopes follow the same flow (ingest → routing → pipeline → storage); inside the pipeline a fixed sequence of stages runs Processing (mask, drop, transform) first and Bucket assignment before Metric extraction, Davis, and Data extraction
- **The default pipeline anti-pattern** — Sending everything through the default pipeline causes query performance, cost, security, and blast radius problems
- **Pipeline design principles** — One pipeline per source type, filter early, differentiate retention, order routing rules from specific to general
- **Processing groups** — Conditional logic within a pipeline: group processors by matching condition to handle multiple data formats without creating separate pipelines. Use groups for different processing; use pipelines for different lifecycle.
- **Security context across scopes** — `dt.security_context` must be set at ingestion; extension metrics require explicit configuration

- **Ingestion vs. query time** — Process at ingestion for permanent actions (drop, mask, route, extract); process at query time for flexible analysis
- 

## Next Steps

---

Continue to **OPIPE-02: Span Processing & Enrichment** to configure OpenPipeline for distributed traces — filtering noisy spans, enriching attributes, and routing traces to dedicated buckets.

---

## References

---

- [OpenPipeline \(DT docs\)](#)
  - [What's new in Dynatrace SaaS 1.346 \(DT docs\)](#) — the `dt.bindplane.*` namespace reservation quoted in §2
  - [Processing in OpenPipeline \(DT docs\)](#)
  - [OpenPipeline data flow \(DT docs\)](#)
  - [Metric extraction stage \(DT docs\)](#)
  - [Davis stage \(DT docs\)](#)
  - [Data extraction stage \(DT docs\)](#)
  - [Primary Grail fields and tags \(DT docs\)](#)
  - [OpenPipeline ingest sources \(DT docs\)](#)
  - [OpenPipeline pipeline groups \(DT docs\)](#)
  - [DQL matcher in OpenPipeline \(DT docs\)](#) — `matchesValue`: *"Accepts wildcard \* at the beginning or at the end of the value"*; the page documents `matchesPhrase`, `matchesValue`, `isNull` and `isNotNull`, and `contains()` / `in()` are rejected by the matcher verifier
  - [Use Grail buckets to partition data \(DT docs\)](#)
  - [Security context for access control \(DT docs\)](#)
  - [SaaS 1.344 release notes \(DT docs\)](#)
  - [OneAgent 1.343 release notes \(DT docs\)](#)
-

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*